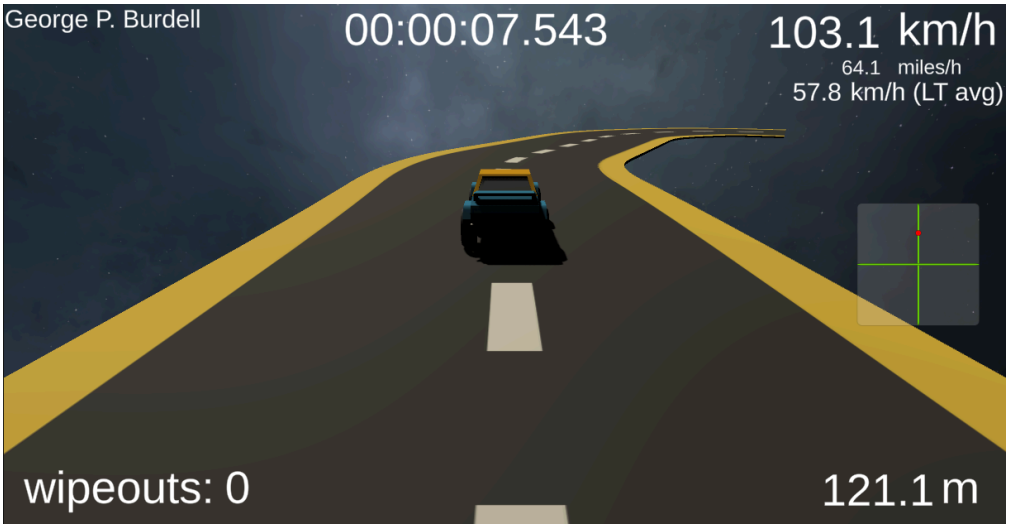
Racetrack Assignment



Credits

The vehicle used is a physically simulated racecar from the Unity Asset Store package *Arcade Car Physics* by *saarg*.

The procedurally generated track system is derived from Seb Lague's *Path-Creator*, a Bézier curve and path editing tool for Unity (https://github.com/SebLague/Path-Creator). The implementation used here has been significantly modified to support real-time procedural generation and runtime sampling within the Racetrack API.

The Fuzzy Logic library used in this assignment is based on the work of `t0chas` on GitHub, inspired by Buckland's *Programming Game AI by Example*. Source repository: https://github.com/t0chas/FuzzyLogic. This framework includes significant modifications to improve API usability, consistency, and readability. It also introduces an alternate fuzzy rule syntax designed to simplify the definition of human-readable rules originally developed by previous OMSCS student, Bob Kerner.

Assignment Work Policy

This is an individual assignment. You must complete it on your own. Do not share your code now or in the future (including posting your solution publicly). Do not view current or former student assignment submissions, and do not use AI code assistants or auto-completion systems to implement this assignment.

Overview

This assignment involves implementing a **Fuzzy Logic Agent** to race on procedurally generated racetracks by designing and tuning fuzzy logic control for a racing vehicle. You will do this by implementing the CSharp file: `Assets/Scripts/GameAIStudentWork/FuzzyVehicle.cs`

Optionally (ungraded), you may also experiment with a **Neural Network Agent** trained to race on the same track.

The goal is to achieve the highest possible sustained speed without crashing, becoming stuck, or turning around.

The vehicle used is a physically simulated racecar from the Unity Asset Store package *Arcade Car Physics* by *saarg*. You can manually drive it by opening the scene:

```
/Scenes/<TrackType>RaceTrackHuman
```

The car exhibits realistic, challenging dynamics. A game controller is strongly recommended.

Fuzzy Logic Library

A Fuzzy Logic library is provided for use in this assignment. It was originally developed by `t0chas` on GitHub and is based on Mat Buckland's *Programming Game AI by Example*. See https://github.com/t0chas/FuzzyLogic.

Documentation for the library is provided in `FuzzyLogAPI_for_Racetrack.pdf`. A detailed example of its usage appears in the scene `FL_3DBall`, where the file `FLBall3DAgent.cs` demonstrates fuzzy control for balancing a ball on an agent's head (adapted from Unity's ML-Agents demo). See `FLBall3DAgentExample.pdf` for a step-by-step walkthrough.

Vehicle API Overview

Your agent class (`FuzzyVehicle.cs` or `NNVehicle.cs` under `Scripts/GameAIStudentWork/`) derives from `AIVehicle`, which controls the Arcade Car.

In `FuzzyVehicle`, you produce `Throttle` and `Steering` in the range $[-1, 1]$ by evaluating fuzzy rules and calling `ApplyFuzzyRules(...)`. Positive throttle accelerates and negative throttle brakes; the car has no reverse gear.

Put this in `UpdateControls()`, which the template overrides. It is called 30 times per second of simulated time, at a fixed rate that does not depend on the rendered frame rate, after the car's state (`Speed`, the `Racetrack` data) has been refreshed. The controls you set are held until the next call. Initialize your fuzzy sets and rules once, in `Start()`.

Core Method

```
ApplyFuzzyRules<T, S>(
    FuzzyRuleSet throttleFRS,
    FuzzyRuleSet steerFRS,
```

```
    FuzzyValueSet fuzzyInput,
    out FuzzyValue[] throttleRuleOutput,
    out FuzzyValue[] steeringRuleOutput,
    ref FuzzyValueSet mergedThrottle,
    ref FuzzyValueSet mergedSteering
)
```

Evaluates both rule sets, merges and defuzzifies the results, and assigns the resulting crisp `Throttle` and `Steering`. This is the only supported way to set vehicle control values.

Developer Helpers

- `HardCodeSteering(f)`, `HardCodeThrottle(f)` — temporary debugging aids. Must be removed before submission.

Read-Only Properties and Helpers

- `Speed` (meters per second), `Speed_kph` (kilometers per hour), `Speed_mph` (miles per hour)
- `transform` — Unity Transform (position, forward, up)
- `Velocity` (Vector3 in meters per second), `AngularVelocity` (Vector3 in radians per second)
- `IsGrounded` — Boolean: true if all wheels are grounded
- `TryGetWheelTelemetry(int idx, out WheelTelemetry t)` — FL=0, FR=1, BL=2, BR=3
- `Racetrack` (read-only) — access to path sampling, lookahead, and Bézier control data
- `ControlDeltaTime` — seconds between `UpdateControls()` calls (1/30 s)

Note. Do not call anything marked `INTERNAL_` or directly manipulate wheels or performance metrics.

Procedurally Generated Tracks

The tracks are based on Seb Lague's `Path-Creator` (https://github.com/SebLague/Path-Creator) but have been heavily modified for real-time procedural generation and runtime sampling.

Agents can access the `Racetrack` property to gather features such as curvature, position by distance, forward direction, etc., for use as crisp input variables in fuzzy rules or NN observations. Refer to `RacetrackAPI.pdf` for details.

Physics and Vector Math Overview

Understanding basic physics relationships is essential for implementing realistic and responsive vehicle behaviors.

Distance–Velocity–Time

The fundamental relationship is:

$$D = V \times T$$

where D is displacement (distance), V is velocity, and T is elapsed time.

`ControlDeltaTime` is the time in seconds between `UpdateControls()` calls (1/30 s). Use it if you compare a value with its value at the previous call, for example to estimate how fast it is changing. (`Time.deltaTime` there is a single physics step, only part of that interval.) To extrapolate a future position, use a constant prediction window:

```
Vector3 futurePos = currentPos + velocity * lookaheadT;
```

Vectors and Points

Unity's `Vector3` represents both points and vectors, though conceptually they are distinct:

- A *point* identifies a location.
- A *vector* represents a direction and magnitude.

For example, $\mathbf{B} - \mathbf{A}$ gives a vector from point A to B.

Adding vectors produces another vector of the same kind. Adding a vector to a point translates it. Adding two points directly is undefined (except for computing averages, such as $(\mathbf{A} + \mathbf{B})/2$).

Useful Unity Functions

- `Vector3.SignedAngle()` — returns the signed angle between two vectors about a given axis.
- `Vector3.Distance()` — computes Euclidean distance.
- `Vector3.Normalize()` — converts a vector to unit length.
- `Vector3.Project()` — projects one vector onto another (useful for projecting vehicle position onto the XZ track plane).

Note. If the velocity vector $\mathbf{V} = (0, 0, 0)$, displacement is zero and direction may be undefined. Guard against division by zero or invalid normalization.

Grading Criteria

First, implement fuzzy logic steering in the CSharp file:
`Assets/Scripts/GameAIStudentWork/FuzzyVehicle.cs`

Your Fuzzy Logic agent must complete four racetracks:

- `RaceTrackFZ` — very curvy
- `WindingRaceTrackFZ` — moderate curves
- `FastSweepersRaceTrackFZ` — high-speed, sweeping turns
- `DragRaceFZ` — perfectly straight

Each run is evaluated over a fixed duration (5 minutes for curvy tracks, 1 minute for `DragRaceFZ`). To earn full credit, the car must achieve the target average speed and complete the duration with no more than one wipeout (flip, stall, or facing the wrong way). Extra credit is available for exceeding speed targets or maintaining zero wipeouts. See `RacingTest.cs` for exact grading thresholds and partial credit rules (implemented as code).

Playmode TestRunner tests are provided for each of the tracks above. Each test reports an estimate of the grade you can expect from the autograder for that section of the rubric.

Fuzzy Logic Implementation Requirements

You must implement control exclusively via human-designed Fuzzy Logic rules:

- Throttle and steering must vary smoothly across continuous ranges.
- Rules must overlap in Degree of Membership (DoM) along the crisp axis to permit interpolation and smooth blending.
- Output sets should have at least three states (possibly more).
- The fuzzy rule set should activate multiple rules simultaneously in most cases.

Rule Coverage Requirement

Your fuzzy rule base must provide full coverage of all valid input values, ensuring that at least one rule is triggered for every possible input on every call to `UpdateControls()`. This prevents situations where no rule fires and guarantees that the system always produces a valid output. Review your rule sets to confirm there are no input gaps or unreachable cases.

Note. You must adhere to the following:

- **DO NOT bypass fuzzy control:** Do not write directly to `Throttle` or `Steering` in `FuzzyVehicle`. Always use `ApplyFuzzyRules(...)`. Remove any hardcoded calls (`HardCodeThrottle`/`HardCodeSteering`) before submission. The autograder will detect them and tests will fail.
- **DO NOT access internal objects:** Never access variables marked `INTERNAL_` or use reflection to modify private state, wheel parameters, or telemetry. Use only the public `Racetrack` API.
- **DO NOT produce invalid outputs:** Ensure your rule base covers all valid inputs so that at least one rule fires on every call to `UpdateControls()`. If defuzzification produces NaN or Infinity, the system will reject and log errors to the output.

Preparing crisp input values for fuzzification should involve minimal conditional logic, if any. Rely on fuzzy reasoning for logical decisions. Input values should generally derive from measurable geometric or kinematic quantities such as relative distances, angles, or speed. A limited exception to the conditional logic requirement is when an `if` statement is used to apply a sign to a vehicle's distance from the center line of the road (though multiplying by the result of `Mathf.Sign(expr)` is preferable). However, hard-coded decision trees to generate abstract "steering intent" or similar intermediate variables are not appropriate. All decision-making logic must reside exclusively within the fuzzy rules.

Static code analysis, runtime validation, and TA inspection will verify compliance. Submissions that circumvent the fuzzy control framework will lose credit, or be refused, even if the autograder initially returns an assessment.

Submission Instructions

Submit your modified `FuzzyVehicle.cs` only. Ensure your HUD name reflects your own identity. Neural Net agents are not submitted but may be discussed on the newsgroup (but don't share the implementation publicly).

Racing Strategy and Tips

Start with intuitive fuzzy rules such as:

> *"If going slow, accelerate." "If too close to the right side, steer left."*

Once basic control is stable, experiment with advanced strategies such as "steering toward a future point on the track" to anticipate turns.

For background on racing line optimization, see https://driver61.com/uni/racing-line/.

Debugging and Visualization

Use Unity's debugging tools:

- `Debug.DrawLine()`, `Debug.DrawRay()` — visualize vectors.
- Inspector variables (`public float DB_throttle`) — live monitoring.
- `vizText` HUD panel — for formatted on-screen output using a `StringBuilder`.

Utility methods such as:

- `RenderFuzzySetAscii<>()`
- `AIVehicle.DiagnosticPrintFuzzyValueSet<>()`
- `AIVehicle.DiagnosticPrintRuleSet<>()`

help visualize fuzzy states for debugging.

Advanced Racing Considerations

The car is rear-wheel drive with a rearward weight bias. At high speeds during turns, oversteer (rear of car coming around) or understeer (front wheels turned but skidding) may occur. Your agent may correct with countersteering, throttle modulation, or braking.

To estimate turning radius (non-sliding conditions):

$$\text{turnRadius} = \frac{\text{wheelbase}}{\tan(\text{steerAngle})}$$

where wheelbase = 2.04 m and track width = 1.32 m (center-to-center) or 1.54 m (outer edges).

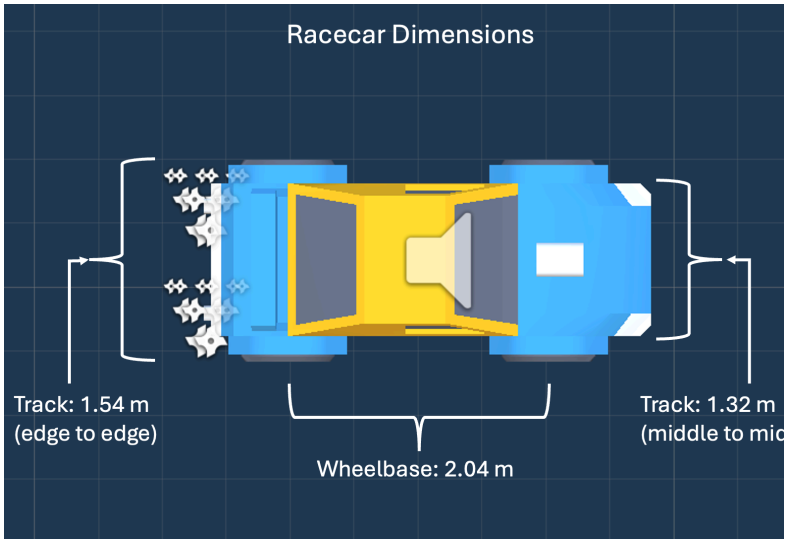


Figure 1: Racecar dimensions: wheelbase and track width.

The center of the turn radius lies perpendicular to the forward vector, on the inside of the curve. Arc length is:

$$\text{arcLength} = \theta_{\text{radians}} \times \text{radius}$$

This geometric reasoning often yields smoother cornering than linear velocity projection.

Performance and Build Notes

Garbage Collection. Because the procedural racetrack rebuilds Bézier segments dynamically, some garbage collection overhead may occur. Incremental GC is enabled to reduce frame spikes.

Builds.

- For gameplay or recording, open `File → Build Settings...` and move your target scene to slot 0.
- Use `Player Settings` to select fullscreen and resolution (e.g., 1920×1080 for video capture).
- For NN training, use the "Training" quality setting with windowed mode (e.g., 160×120 resolution).
- Multi-core processors can train several builds simultaneously.